

1 Environment models

Of the three forces the propagator integrates — thrust, gravity, and drag (??) — the last two are the environment's to answer. `sim::Environment` is the container through which the flight dynamics see the outside world: it holds one live instance of each of three pluggable physics models — a `GravityModel`, an `AtmosphericModel`, and a `GeoidModel` `sim/Environment.h:27` — selected by display-name string and defaulting to "Constant Gravity" plus "Constant Atmosphere" `sim/Environment.h:31`. The container itself is deliberately thin: a 123-line header-only class `sim/Environment.h` whose whole job is to own the selected models and hand them out. The chapter walks the container first, then each model family. The centerpiece is the US Standard Atmosphere 1976 — a closed-form, seven-layer implementation validated to 0.1% against the published report tables — and the chapter closes with the two deliberately simple atmospheres and the one model that exists but is consumed by nothing: wind.

One asymmetry frames everything that follows. The environment can produce six physical quantities — a gravity vector and five atmospheric properties — but the flight dynamics consume exactly two of them: gravitational acceleration `model/RocketModel.cpp:77` and air density `model/RocketModel.cpp:86`. Pressure, temperature, speed of sound, and dynamic viscosity are computed, oracle-validated, and unconsumed — they are the natural inputs for a future Mach-dependent drag coefficient and Reynolds-dependent skin friction (??), and until that machinery lands they are dormant outputs behind a live interface. [Figure 1](#) shows the arrangement.

1.1 The container: registries, selection, and injection

`Environment` is singleton-by-construction per application: its copy and move constructors and assignment operators are all deleted `sim/Environment.h:37`, and the one instance is created by the QtRocket controller at startup `QtRocket.cpp:43`. Internally it keeps two `std::maps` from display-name string to `std::shared_ptr` — three atmosphere keys ("Constant Atmosphere", "US Standard 1976", "Vacuum") `sim/Environment.h:103` and two gravity keys ("Constant Gravity", "Spherical Gravity") `sim/Environment.h:108` — plus a selection string per registry `sim/Environment.h:112` and the single eagerly constructed geoid `sim/Environment.h:117`. The maps are declared with *null* pointers; the constructor immediately calls both setters so that the two default selections point at live instances from the first moment `sim/Environment.h:31`, and the defaults' observable behavior — sea-level constants at every altitude, constant (0, 0, $-g_0$) gravity — is pinned by `DefaultsToConstantAtmosphereAndConstantGravity` `sim/tests/EnvironmentTests.cpp:45`.

Selection is a string comparison, nothing more. `setAtmosphereModel` is an if/else-if chain over the three known names `sim/Environment.h:74`; each matching branch records the name and builds a fresh instance — `atmosphereModels[atmosphereModel].reset(new sim::USStandardAtmosphere);` `sim/Environment.h:84` — and no `else` branch exists, so an unknown name changes nothing. The gravity setter is symmetric `sim/Environment.h:60`, with the spherical branch injecting the geoid into the new model's constructor `sim/Environment.h:70`. Retrieval is a map lookup on the current selection string `sim/Environment.h:93`, and enumeration copies the map keys out with `std::transform` `sim/Environment.h:42` — `std::map` iteration order makes both lists alphabetical, and a test pins the exact names and their order `sim/tests/EnvironmentTests.cpp:58`.

Unknown names are no-ops; known names are fresh instances

Two selection semantics are easy to miss and both are load-bearing. First, the setters' if/else-if chains have no fallback branch `sim/Environment.h:60` `sim/Environment.h:74`: an unrecognized

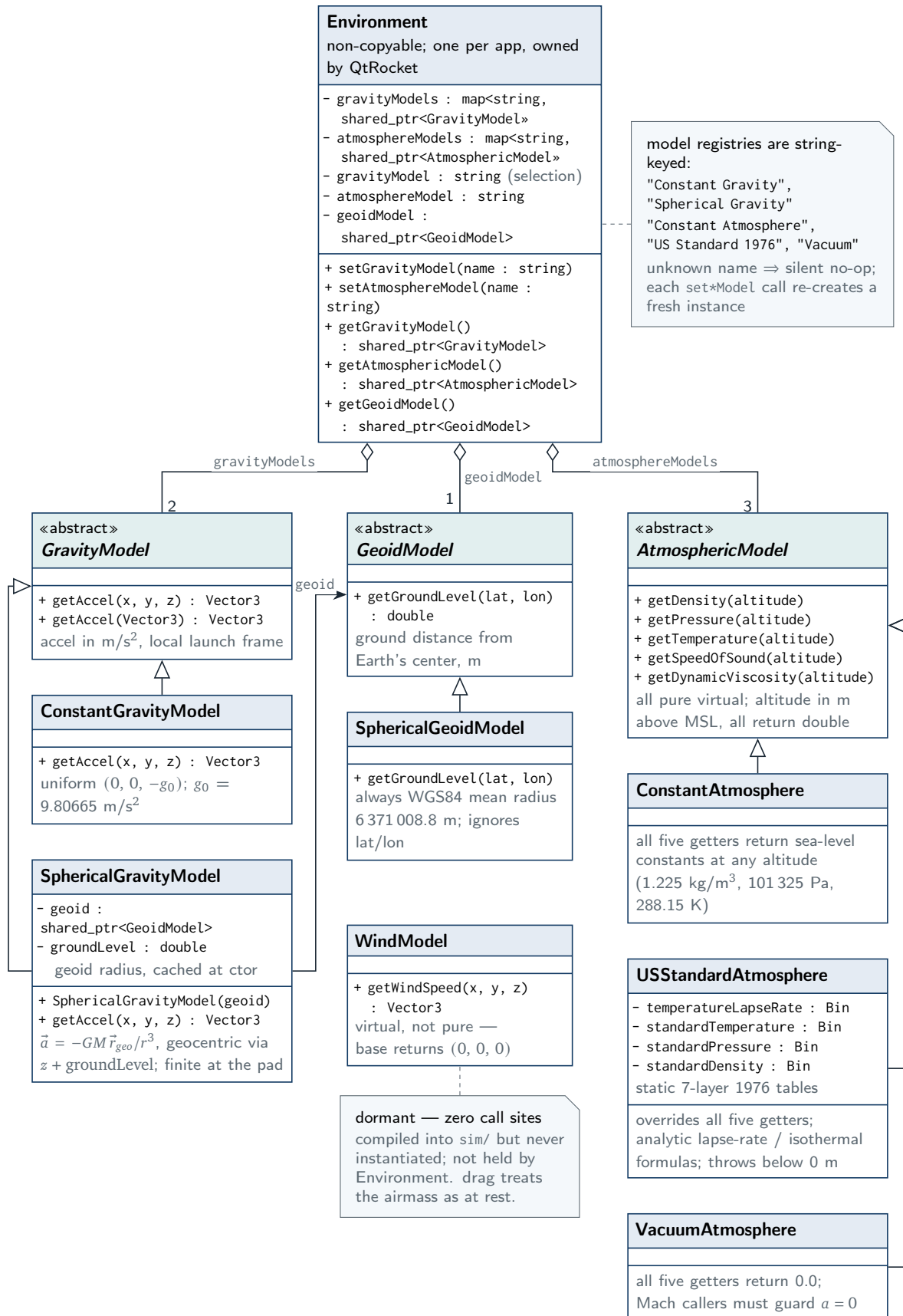


Figure 1. The environment subsystem. Environment owns one selected instance behind each of its two string-keyed registries plus the single geoid `sim/Environment.h:28`; the abstract interfaces fan out to the concrete models named in the registries. The consumption chain runs from the QtRocket controller, which owns the one shared instance, through the Propagator into `RocketModel::getForces`, which consumes exactly two quantities: gravitational acceleration and air density. The atmosphere's other four properties are computed and unconsumed, and `WindModel` stands entirely outside the container.

name silently leaves the current selection untouched, down to pointer identity — the tests assert the *same* `shared_ptr` remains selected after a bogus request `sim/tests/EnvironmentTests.cpp:99`. Second, every successful `set` call constructs a new model object, even when re-selecting the current name. All models are stateless (the spherical gravity model’s cached ground radius reconstructs identically), so this is harmless today — but a client that caches a fetched `shared_ptr` keeps the *old* instance alive and stops observing selection changes. Correct usage is to re-fetch per use, which is exactly what `RocketModel::getForces` does on every call `model/RocketModel.cpp:75` `model/RocketModel.cpp:83`. One related hazard: `getAtmosphericModel()` looks up the selection with `std::map::operator[]` `sim/Environment.h:95`, which would silently insert a null entry if the selection string were ever not a key — unreachable through today’s public API, but armed the day a setter branch is added without a matching registry entry.

Silent rejection at this layer is safe because the layer above never guesses at names. The registries are, in the GUI’s own words, “the source of truth for valid names” `gui/SimOptionsTab.cpp:31`: the GUI populates its combo boxes from the `getAvailable*Models` lists `gui/SimOptionsTab.cpp:32`, the CLI validates a requested name against them before calling the setter and reports `ERR` on a miss `cli/Repl.cpp:585` `cli/Repl.cpp:612` (`??`), and the CLI’s prompt-state defaults carry a “matches Environment default” comment so the two layers cannot drift silently `cli/Repl.h:45`.

Environment by injection, not by lookup

The environment reaches the physics by parameter, never by global lookup. `QtRocket` owns the single shared instance (`QtRocket.h:82`, exposed at `QtRocket.h:34`); each `Propagator` receives it at construction `sim/Propagator.h:44` and dereferences it into the force callback on every integrator stage — `object->getForces(t, state, rate, *environment)` `sim/Propagator.cpp:41`; and `RocketModel::getForces` takes it as `sim::Environment&` `model/RocketModel.cpp:68`. Front ends mutate the same shared instance in place — the GUI’s change slots write through it precisely so that changing the atmosphere cannot clobber the gravity selection `gui/SimOptionsTab.cpp:91` — and every holder observes the change on its next fetch.

1.2 Gravity

The constant model is one line of physics: at any position, `return Vector3(0.0, 0.0, -utils::math::Constants::g0)`; `sim/ConstantGravityModel.h:18`, a uniform field of 9.80665 m/s^2 pointing down the world z axis. The value is not arbitrary: $g_0 = 9.80665 \text{ m/s}^2$ `utils/math/Constants.h:10` is *standard gravity*, the conventional sea-level weight acceleration, and it quietly bundles more than gravitation — see the box below.

Inverse-square gravity, and why GM is one constant

Newton’s law of gravitation for a spherically symmetric Earth gives

$$\vec{a} = -\frac{GM}{r^3} \vec{r},$$

where $\vec{r}$ is the geocentric position and $r = |\vec{r}|$. The product GM — the *gravitational parameter* — is kept as a single constant because it is known far more precisely than either factor alone: spacecraft tracking measures GM directly, while G is among the least precisely known fundamental constants. For Earth, $GM = 3.986004418 \times 10^{14} \text{ m}^3/\text{s}^2$. At the surface this gives $GM/R^2 \approx 9.820 \text{ m/s}^2$ — about 0.13% *above* standard gravity’s 9.80665 , because the conven-

tional value folds in the centrifugal relief of Earth’s rotation, which a purely gravitational model does not.

`SphericalGravityModel` implements exactly that law, with one preparatory step. The simulation state lives in a local launch frame — origin at the pad, z up, ground at $z = 0$ `sim/SphericalGravityModel.h:19` — so before applying an inverse-square law the model must re-express the position geocentrically. Its constructor caches the launch site’s ground radius once, because the radius is constant for a site and `getAccel` runs on every integrator stage `sim/SphericalGravityModel.cpp:18`; the geoid is asked at placeholder coordinates $(0, 0)$ “until a real launch site is plumbed in” `sim/SphericalGravityModel.cpp:19`. The acceleration itself is five lines `sim/SphericalGravityModel.cpp:33`: in the code’s names, with $gz = z + \text{groundLevel}$ and $\vec{r}_{\text{geo}} = (x, y, gz)$,

$$\vec{a} = -\frac{\text{earthGM}}{r^3} \vec{r}_{\text{geo}}, \quad r = |\vec{r}_{\text{geo}}|, \quad \text{earthGM} = 3.986004418 \times 10^{14} \text{ m}^3/\text{s}^2 \quad (1)$$

`sim/SphericalGravityModel.cpp:36` `utils/math/Constants.h:26`, written as a single scalar `factor = -earthGM/(r2 · r)` so the vector multiply distributes over components. Tests pin the surface magnitude to GM/R^2 within 10^{-12} relative error `sim/tests/EnvironmentTests.cpp:123`, the $GM/(R+z)^2$ falloff at 10 km `sim/tests/SphericalGeoidAndGravityTests.cpp:67`, and the small inward horizontal component one kilometer downrange — the vector points at Earth’s center, not straight down `sim/tests/SphericalGeoidAndGravityTests.cpp:82`.

The ground-radius offset fixed a real hang

The line `const double gz = z + groundLevel;` is not a refinement; it is the fix for a located defect. An earlier `SphericalGravityModel` treated the pad-relative coordinates as geocentric and computed GM/r^3 at the pad’s $r = 0$ — producing NaN and hanging the propagator loop in an infinite iteration (the F1 defect recorded in the test file’s header `sim/tests/SphericalGeoidAndGravityTests.cpp:1`). Today the invariant is stated where it binds — “ $|rvec| \geq \text{groundLevel}$ so it’s never 0” `sim/SphericalGravityModel.cpp:32` — and defended twice: the `FiniteAtLaunchOrigin` regression test asserts finite acceleration at the exact origin `sim/tests/SphericalGeoidAndGravityTests.cpp:48`, and the propagator’s `NonFiniteState` guard aborts rather than loops if a NaN ever reaches the state again `sim/Propagator.cpp:94` (??).

The ground radius comes from the geoid. A geoid, in theory, is the equipotential surface of Earth’s gravity field — “mean sea level” extended under the continents — and the interface asks it a single question: how far is the ground from Earth’s center at a given latitude and longitude `sim/GeoidModel.h:14`. The only implementation, `SphericalGeoidModel`, ignores both arguments and returns `meanEarthRadiusWGS84 = 6 371 008.8 m` `sim/SphericalGeoidModel.cpp:18` `utils/math/Constants.h:24` — the WGS84 (World Geodetic System 1984) mean of Earth’s polar and equatorial radii `sim/SphericalGeoidModel.h:10`. There is no `setGeoidModel`; `Environment` constructs the one instance in-class and notes “Only one geoid today; a registry can follow if needed” `sim/Environment.h:115`. Constancy across latitudes is itself pinned by test `sim/tests/SphericalGeoidAndGravityTests.cpp:38`.

End to end, the two gravity models agree the way the physics box predicts: an integration test flies the same rocket under both and requires the apogees to land within 5% — spherical $g \approx 9.82 \text{ m/s}^2$ near the surface versus the constant model’s 9.80665 `tests/PhysicsIntegrationTests.cpp:331`.

1.3 The US Standard Atmosphere 1976

`USStandardAtmosphere` — registry name "US Standard 1976" — is the subsystem’s centerpiece: a closed-form implementation of the 1976 US Standard Atmosphere’s lower layers, from sea level through the mesosphere `sim/USStandardAtmosphere.cpp`. The reference document whose tables anchor both the implementation and its test oracles is bundled in the repository `docs/us-standard-atmosphere_st76-1562_noaa.pdf`. The physics behind the formulas fits in one box.

Two relations govern a standard atmosphere

A static column of air must support its own weight: the pressure drop across a horizontal slab of thickness dh equals the slab’s weight per unit area,

$$\frac{dP}{dh} = -\rho g \quad (\text{hydrostatic equilibrium}).$$

Density, pressure, and temperature are tied by the ideal gas law, $P = \rho \frac{R^*}{M} T$, with R^* the universal gas constant and M the mean molar mass of dry air. Substituting one into the other eliminates ρ :

$$\frac{dP}{P} = -\frac{g_0 M}{R^* T(h)} dh.$$

The 1976 standard makes this integrable in closed form by *prescribing* $T(h)$ as piecewise linear: seven layers, each with a constant lapse rate $L_b = dT/dh$. Two layer types result. In a *gradient* layer ($L_b \neq 0$) the integral yields a power law, $P = P_b [T(h)/T_b]^{-g_0 M / (R^* L_b)}$; in an *isothermal* layer ($L_b = 0$) it yields an exponential, $P = P_b e^{-g_0 M (h-h_b) / (R^* T_b)}$. Density then follows from the gas law — dividing the pressure law by one more factor of $T(h)/T_b$ — which is why the density exponent is the pressure exponent minus one, and why the isothermal branches of P and ρ share the same exponential. Physically: pressure falls with height because less air sits above you, roughly exponentially with an 8 km scale height (ρ drops ~63% by 11 km and 99.99% by 71 km); temperature zig-zags — the troposphere cools with altitude, the stratosphere warms because ozone absorbs ultraviolet, the mesosphere cools again.

The constants. The formulas run on the 1976 standard’s own constants, not modern CODATA values: `Rstar = 8.31432 J mol-1 K-1`, `airMolarMass = 0.0289644 kg/mol`, and `g0 = 9.80665 m/s2` `utils/math/Constants.h:9`, plus `gamma = 1.4` for the speed of sound `utils/math/Constants.h:15` and the Sutherland pair `beta = 1.458 × 10-6 kg s-1 m-1 K-1/2` — “empirically measured,” the header notes, per the 1976 report — and `S = 110.4 K` `utils/math/Constants.h:19`. Three neighbors in the same header are dead: `standardTemperature`, `standardDensity`, and `earthGM_km` have no references outside their definitions `utils/math/Constants.h:22` — the first two share names with `USStandardAtmosphere`’s static tables, which are distinct entities and are what the formulas actually read.

The lookup primitive. Each per-layer quantity lives in a `utils::Bin`, a small piecewise-bin table: “each inserted key is a bin’s lower bound” and `operator[]` returns the value of the bin containing the query key `utils/Bin.h:15`. Insertion appends and re-sorts `utils/Bin.cpp:41`; lookup walks forward until a bin base exceeds the key, and “falling off the end means key is in the last bin” `utils/Bin.cpp:57` — so the topmost layer extends upward without bound. Keys below the lowest base throw `std::out_of_range` rather than silently clamping `utils/Bin.cpp:52`. A companion `getBinBase` returns the containing bin’s lower-bound key instead `utils/Bin.cpp:72`; the formulas below need both the layer’s anchored value and its base altitude.

The tables. Four static `utils::Bin` members — `temperatureLapseRate`, `standardTemperature`, `standardDensity`, `standardPressure` `sim/USStandardAtmosphere.h:28` — are populated once at static-initialization time `sim/USStandardAtmosphere.cpp:78` from the seven layer anchors in [Table 1](#) `sim/USStandardAtmosphere.cpp:21`. The layer-base pressures and densities are hard-coded from the report rather than derived recursively from the sea-level values, so each layer’s formula is anchored directly to published numbers rather than to accumulated floating-point error.

h_b (m)	T_b (K)	coded rate (K/m)	standard L_b (K/km)	P_b (Pa)	ρ_b (kg/m ³)
0	288.15	+0.0065	−6.5	101325	1.225
11000	216.65	0.0	0	22632.1	0.36391
20000	216.65	−0.001	+1.0	5474.89	0.08803
32000	228.65	−0.0028	+2.8	868.02	0.01322
47000	270.65	0.0	0	110.91	0.00143
51000	270.65	+0.0028	−2.8	66.94	0.00086
71000	214.65	+0.002	−2.0	3.96	0.000064

Table 1. The seven layers as coded `sim/USStandardAtmosphere.cpp:21`: base altitude, base temperature, the stored `temperatureLapseRate`, the standard’s lapse rate $L_b = dT/dh$ it corresponds to, and the hard-coded pressure and density anchors. The coded rate is the *negative* of L_b — positive where temperature falls with altitude — and every formula in the implementation uses that flipped convention consistently.

Sign convention. The stored `temperatureLapseRate` is the negative of the standard’s L_b : the troposphere is coded +0.0065 K/m — temperature *falls* 6.5 K per kilometer — while the warming layers based at 20 and 32 km are coded negative `sim/USStandardAtmosphere.cpp:36`. The convention is harmless because all three formulas use it consistently; with $L = -L_b$ the coded power law below is algebraically identical to the report’s equation (33a).

Temperature. With $h_b = \text{standardTemperature.getBinBase}(h)$, temperature is linear within the layer `sim/USStandardAtmosphere.cpp:114`:

$$T(h) = \text{standardTemperature}[h] - (h - h_b) \cdot \text{temperatureLapseRate}[h]. \quad (2)$$

Pressure. `getPressure` branches on whether the layer is isothermal, testing `floatingPointEqual_j` (`temperatureLapseRate[altitude]`, `0.0`) `sim/USStandardAtmosphere.cpp:123` — a ULP-scaled floating-point comparison `utils/math/UtilityMathFunctions.h:21` that is exact here only because the two zero rates are stored as literal `0.0` `sim/USStandardAtmosphere.cpp:40`. The isothermal branch (layers based at 11 and 47 km) is the exponential `sim/USStandardAtmosphere.cpp:125`:

$$P(h) = \text{standardPressure}[h] \exp\left(\frac{-g_0 \cdot \text{airMolarMass} \cdot (h - h_b)}{R_{\text{star}} \cdot \text{standardTemperature}[h]}\right), \quad (3)$$

and the gradient branch is the power law of [Listing 1](#): with $T_b = \text{standardTemperature}[h]$, $L = \text{temperatureLapseRate}[h]$, and $P_b = \text{standardPressure}[h]$,

$$P(h) = P_b \cdot \text{base}^{\text{exponent}}, \quad \text{base} = \frac{T_b - L(h - h_b)}{T_b}, \quad \text{exponent} = \frac{g_0 \cdot \text{airMolarMass}}{R_{\text{star}} \cdot L}. \quad (4)$$

Note that `base` is just $T(h)/T_b$ — the ratio of local to layer-base temperature — so [Eq. \(4\)](#) is the physics box’s power law in the code’s flipped-sign convention.

```

double base = (standardTemperature[altitude] - temperatureLapseRate[altitude] *
               (altitude - standardPressure.getBinBase(altitude))) / standardTemperature[altitude];

double exponent = (Constants::g0 * Constants::airMolarMass) /
                  (Constants::Rstar * temperatureLapseRate[altitude]);

return standardPressure[altitude] * std::pow(base, exponent);

```

Listing 1. The gradient-layer pressure power law as coded `sim/USStandardAtmosphere.cpp:132`: base is the temperature ratio $T(h)/T_b$, and four Bin lookups anchor the layer.

Density. `getDensity` is structurally identical — the same isothermal test, the same two branches — reading the `standardDensity` anchors instead of `standardPressure` `sim/USStandardAtmosphere.cpp:93`, with exactly one difference: the gradient-layer exponent subtracts one `sim/USStandardAtmosphere.cpp:107`,

$$\rho(h) = \rho_b \cdot \text{base}^{\frac{g_0 \cdot \text{airMolarMass}}{R_{\text{star}} \cdot L} - 1}. \quad (5)$$

The -1 is the ideal gas law’s doing: $\rho = PM/(R \cdot T)$, and dividing the pressure power law by one extra factor of $T(h)/T_b$ lowers the exponent by exactly one. In the isothermal branch T is constant, so density and pressure share the same exponential form. Density is the one atmospheric number flight actually consumes, entering the drag force $\vec{F}_d = -\frac{1}{2}\rho |\vec{v}| C_d A \vec{v}$ at `model/RocketModel.cpp:86` (????).

Auxiliary properties. Speed of sound is the perfect-gas formula `sim/USStandardAtmosphere.cpp:143` and dynamic viscosity is Sutherland’s law `sim/USStandardAtmosphere.cpp:150`, both driven by Eq. (2):

$$a(h) = \sqrt{\frac{\text{gamma} \cdot R_{\text{star}} \cdot T(h)}{\text{airMolarMass}}}, \quad \mu(h) = \frac{\text{beta} \cdot T(h)^{3/2}}{T(h) + S}. \quad (6)$$

At 288.15 K these evaluate to 340.29 m/s and 1.7894×10^{-5} Pa s — matching the constant atmosphere’s hard-coded 340.294 and 1.78938×10^{-5} `sim/ConstantAtmosphere.h:19`. Both are the 1976 report’s own auxiliary formulas, and both are unconsumed: a whole-tree search finds no caller of `getSpeedOfSound` or `getDynamicViscosity` outside the models and their tests — no Mach number, no Reynolds number, nothing (????).

Validated against the report, not against itself

The oracle suite compares the implementation to the published NOAA tables, never to its own output, and every assertion is a *relative* bound: `EXPECT_NEAR(computed / published, 1.0, _tol)`. Pressure and temperature must match to 0.1% at twenty altitudes from 0 to 80 000 m `sim/tests/USStandardAtmosphereTests.cpp:36` `sim/tests/USStandardAtmosphereTests.cpp:64`; density must match to 0.1% at thirteen altitudes `sim/tests/USStandardAtmosphereTests.cpp:10` and to 0.5% at seven more — 8–10 km and 50–80 km, where the file comments record a “slight deviation of calculated density from the given density in the report table” `sim/tests/USStandardAtmosphereTests.cpp:24`. The 80 km points sit in the *extrapolated* last bin (base 71 000 m) and still meet the 0.1% pressure oracle. One loose end: `getDensity` still carries an open “@todo Verify against the 1976 US Standard Atmosphere / NOAA paper” `sim/USStandardAtmosphere.h:18` — verification that now exists.

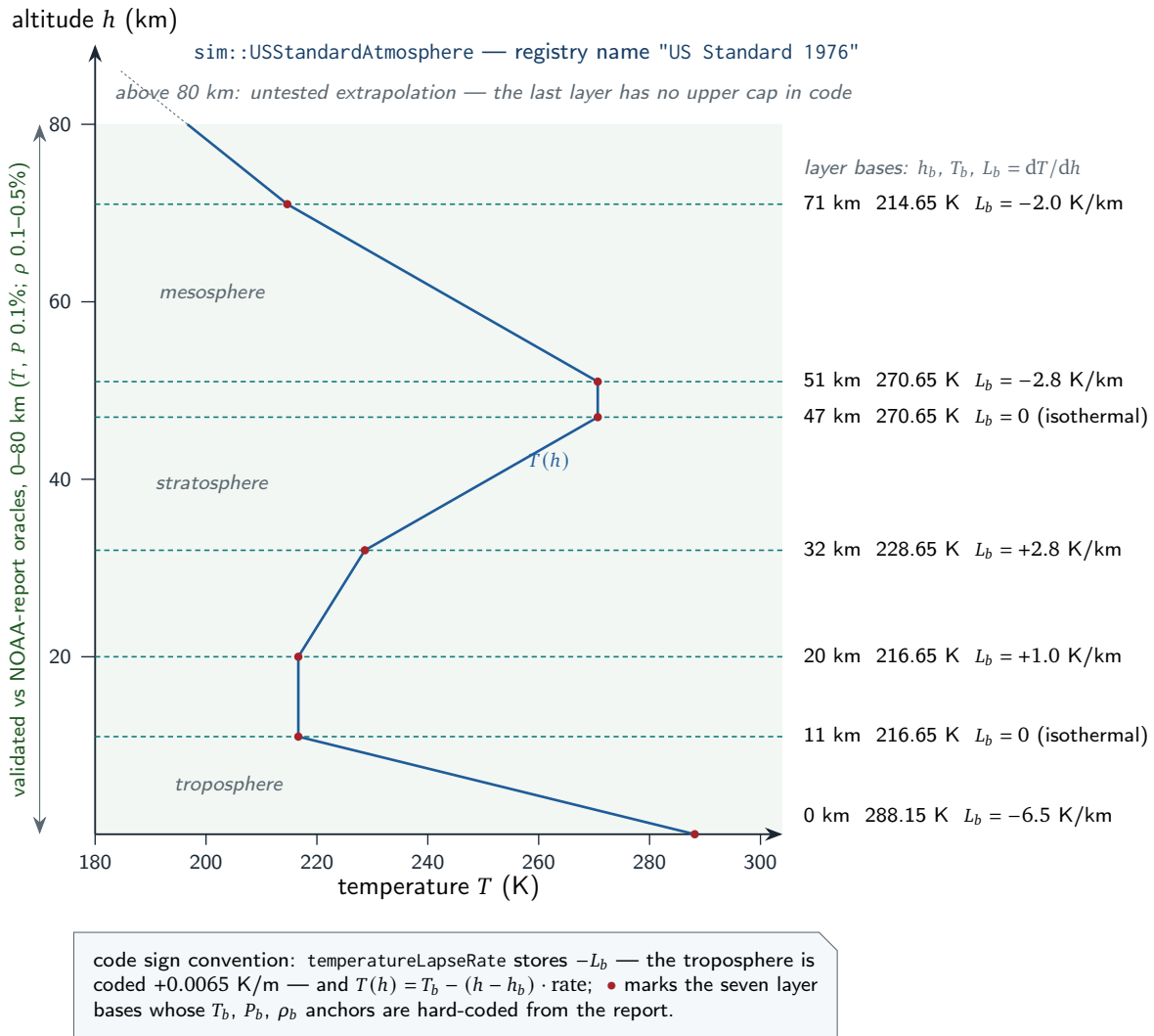


Figure 2. The temperature profile $T(h)$ of the US Standard Atmosphere 1976 as coded: piecewise linear between the seven layer bases whose temperature, pressure, and density anchors are hard-coded from the NOAA report `sim/USStandardAtmosphere.cpp:21`. The shaded band marks the oracle-validated range, 0–80 km; above it the last layer's formulas extrapolate without bound. The bottom note records the sign convention: the stored `temperatureLapseRate` is the negative of the standard's L_b .

The negative-altitude trap, and who guards it

The model's domain is altitude ≥ 0 : any query below the lowest bin base propagates `std::out_of_range` from `utils::Bin::operator[]` `utils/Bin.cpp:52`. This was a real crash, not a hypothetical — integrator trial states dip below $z = 0$ near landing, and before the fix a full flight under "US Standard 1976" died there. The guard lives on the *consumer* side: `RocketModel::getForces` clamps — "Clamp altitude to ≥ 0 : trial states crossing $z=0$ fall outside the atmosphere models' domain" `model/RocketModel.cpp:84` — and the regression is pinned end-to-end by `USStandardAtmosphereCompletesWithoutCrash` `tests/PhysicsIntegrationTests.cpp:277`. The trap hides behind a shared interface — the constant and vacuum models are total functions over the same inputs (Section 1.4), so a caller tested only against those two would pass every test and still crash here. Any new consumer of `AtmosphericModel` owes the same clamp.

Two further edges are honest limitations rather than defects. Above the last layer base there is no cap: the standard's tables end at 84 852 m and switch to a kinetic model at 86 km, but the code's 71 km layer extrapolates its closed forms to arbitrary altitude because the last `Bin` is unbounded `utils/Bin.cpp:57` — accurate at 80 km per the oracles, untested and increasingly wrong beyond. And the implementation feeds geometric altitude straight into tables the standard defines in *geopotential* height — no conversion $H = zR/(R + z)$ exists anywhere in the tree. Below 11 km the difference is under 0.2%, which is negligible at model-rocket altitudes; at 50–80 km it is part of what the 0.5% density tolerances absorb. Both belong to the consolidated inventory in ??.

1.4 The constant and vacuum atmospheres

The other two atmospheres are deliberately trivial, and each earns its keep in testing. `ConstantAtmosphere` — the default selection — returns the five sea-level standard values at every altitude: density 1.225 kg/m^3 , pressure $101\,325 \text{ Pa}$, temperature 288.15 K , speed of sound 340.294 m/s , and dynamic viscosity $1.78938 \times 10^{-5} \text{ Pa s}$ `sim/ConstantAtmosphere.h:15`. For flights that top out a few hundred meters up, a constant-density column is a fine first approximation, and it makes hand-checkable physics: the `TerminalVelocityForceBalance` integration test pulls ρ and g from the live default models and verifies the net force vanishes at $v_t = \sqrt{2mg/(\rho C_d A)}$ `tests/PhysicsIntegrationTests.cpp:246`.

`VacuumAtmosphere` returns zero for all five properties `sim/VacuumAtmosphere.h:17`: "drag vanishes and the flight reduces to thrust + gravity," making it the baseline that isolates integrator behavior from the aero model `sim/VacuumAtmosphere.h:9`. That role is exercised constantly. `DragReducesApogeeVersusVacuum` flies the same rocket under vacuum and constant atmosphere and requires the drag apogee to fall below 75% of the vacuum apogee `tests/PhysicsIntegrationTests.cpp:227`; the design-matrix sweep flies each reference airframe's motor ladder twice — once under "Vacuum" with $C_d = 0$, once under "US Standard 1976" with $C_d = 0.75$ — and asserts drag always costs altitude, under both integrators `tests/DesignMatrixTests.cpp:542` `tests/DesignMatrixTests.cpp:559` (one motor per impulse class in the fast suite, the complete ladder under the heavy label; ??). One forward-looking wrinkle is documented in place: a vacuum has no sound, so `getSpeedOfSound` returns 0 and "callers computing a Mach number must guard against 0" `sim/VacuumAtmosphere.h:21` — a division hazard that becomes live the day Mach-dependent drag arrives. Both simple models are total functions over pathological inputs; the test grid includes -100 m and NaN `sim/tests/SimpleAtmosphereTests.cpp:13`.

1.5 Wind: a dormant seam

`WindModel` completes the environment’s inventory, and it is the subsystem’s one fully dormant class in the series’ strict sense: built, compiled, and consumed by nothing. It is a concrete base class — virtual but not pure, because the base is the “no wind” model — with a single method `virtual Vector3 getWindSpeed(double x, double y, double z);` `sim/WindModel.h:16` whose implementation returns `Vector3(0.0, 0.0, 0.0)` `sim/WindModel.cpp:16`. It is compiled into the `sim` library `sim/CMakeLists.txt:24`, and that CMake listing plus its own two source files are the only places its name appears in the tree: no subclasses, no `Environment` member, no registry entry, and zero call sites for `getWindSpeed`. Consequently the drag law computes drag from the vehicle’s inertial velocity alone `model/RocketModel.cpp:88` — the air mass is implicitly at rest.

Plumbing the seam is a bounded job, and the container shows the pattern. Aerodynamically, drag opposes the *air-relative* velocity $\vec{v} - \vec{w}$, so consuming wind means: a wind registry and setter in `Environment` mirroring the other two registries `sim/Environment.h:103`; one subtraction in `RocketModel::getForces`, replacing `velocity` with the relative velocity before the drag product `model/RocketModel.cpp:88`; and a selection surface in the front ends. Under today’s 3-DOF point mass a steady wind would add only a lateral drag drift; the payoff that motivates wind in a rocket simulator — weathercocking into the breeze off the launch rod — additionally requires the angle-of-attack and torque machinery of ???. The consolidated inventory in ??? carries both entries.

The pattern across this chapter is the document’s in miniature. What flight consumes from the environment is two numbers per integrator stage — one gravity vector, one density — and everything behind those numbers is small, closed-form, and validated well past the consumed surface: five-decade pressure oracles for a drag law that only reads ρ , a speed of sound nothing divides by yet, a wind vector nothing subtracts. When Mach-dependent drag or wind arrives, the environment will not need to grow; its consumers will.